



Documento de cierre técnico: resumen ejecutivo del proyecto y registro de las lecciones aprendidas técnicas.

Aprendiz – Juan Pablo Mejia Vargas CC 1012359571

Servicio Nacional de Aprendizaje SENA

Centro de Formación en Diseño, Confección y Moda

Tecnología en Análisis y Desarrollo de Software – ADSO

Ficha: 2977373



1. RESUMEN EJECUTIVO

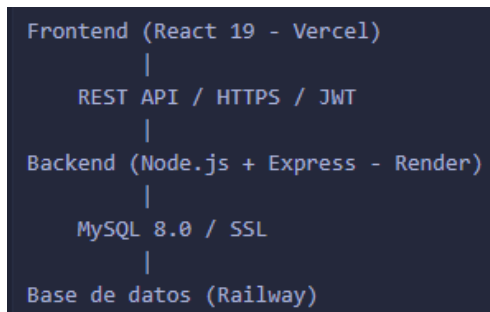
1.1 Descripción del Proyecto

Segura-Mente App es una aplicación web de gestión de salud mental desarrollada como proyecto de etapa productiva del programa de Análisis y Desarrollo de Software del SENA. La plataforma permite el registro de usuarios, autenticación segura y agendamiento de citas entre clientes y psicólogos.

1.2 Alcance Ejecutado

Modulo	Estado	Descripción
Autenticación y registro	Completado	Registro con validaciones, login con JWT, control de sesión por inactividad
Gestión de usuarios	Completado	CRUD completo por parte del administrador
Agendamiento de citas	Completado	Calendario, selección de psicólogo y horario, creación, modificación y cancelación
Panel del psicólogo	Completado	Vista de citas asignadas, cancelación con notificación al cliente
Despliegue en producción	Completado	Frontend en Vercel, backend en Render, base de datos en Railway
Pruebas automatizadas	Completado	102 casos de prueba implementados con Cypress

1.3 Arquitectura Implementada





1.4 Métricas del Proyecto

Indicador	Valor
Total de módulos funcionales	5
Endpoints de la API REST	11
Tablas en la base de datos	2
Casos de prueba automatizados	102
Casos de prueba exitosos	102
Commits en el repositorio	25+
Ambiente de despliegue	Producción (nube)
URL de producción	https://segura-mente-app-final.vercel.app

1.5 Funcionalidades Entregadas

Completamente funcionales:

- Registro de usuarios (Cliente y Psicólogo/empleador) con validaciones completas
- Inicio de sesión con token JWT
- Control automático de sesión por inactividad (5 minutos con advertencia de 1 minuto)
- Dashboard diferenciado por rol de usuario
- Agendamiento de citas con calendario navegable
- Modificación y cancelación de citas por el cliente
- Panel de citas para el psicólogo con opción de cancelación
- Notificación visual al cliente (en su dashboard) cuando el psicólogo cancela una cita
- Gestión completa de usuarios por el administrador (CRUD)
- Despliegue con CI/CD automático desde GitHub

Pendientes de implementación futura:

- Envío de correos electrónicos (SMTP no configurado en producción)
- Recuperación de contraseña por email
- Notificaciones en tiempo real



2. REGISTRO DE LECCIONES APRENDIDAS TECNICAS

Lección 1 - Operaciones bloqueantes en Node.js

El endpoint de registro tardaba entre 30 y 90 segundos en responder porque el servidor esperaba la respuesta del servidor SMTP antes de retornar al cliente. Esto degradaba completamente la experiencia de usuario. La causa se debía a que el comando `await sendVerificationEmail()` bloqueaba el hilo principal de Node.js mientras esperaba la respuesta del servidor de correo, que finalmente fallaba por timeout (30 segundos configurados en nodemailer).

Para solucionarlo se convirtió el envío de correo en una operación no bloqueante usando `.then().catch()` en lugar de `await`. El servidor responde al cliente inmediatamente y el intento de envío de correo ocurre en segundo plano. Los timeouts del transporter SMTP se redujeron de 30 a 5 segundos. Como un aprendizaje se tendría que en APIs REST, las operaciones secundarias (notificaciones, emails, logs externos) nunca deben bloquear la respuesta principal. Usar el patrón "fire and forget" para operaciones que no son críticas para el flujo del usuario.

Lección 2 - comparación de strings con caracteres especiales

La vista del psicólogo mostraba el calendario de agendamiento (vista de cliente) en lugar del panel de citas, a pesar de que el usuario tenía el tipo correcto en la base de datos.

Debido a que la comparación `user.tipoUsuario === 'Psicólogo/empleador'` fallaba por variaciones en tildes, mayúsculas o encoding del carácter ó al ser almacenado y recuperado de `localStorage`.

Un hash de codificación diferente entre UTF-8 y lo almacenado causaba que la comparación estricta retornara `false`.



Como solución se reemplazo la comparación estricta por una búsqueda flexible: `tipo.toLowerCase().includes('empleado') || tipo.toLowerCase().includes('psic')`. Esto hace la validación insensible a mayúsculas, tildes y variaciones de escritura.

Como aprendizaje podemos decir que las comparaciones de strings con datos provenientes de formularios o bases de datos deben ser tolerantes a variaciones. Usar `.toLowerCase()` y `.includes()` en lugar de `===` cuando el valor puede tener variaciones de encoding o escritura.

Lección 3 - Variables de entorno en React (tiempo de compilación)

Las peticiones al backend llegaban con una URL malformada como `https://vercel.app/mi-backend.onrender.com/api/auth/login`, lo que generaba errores 404 y 405 en lugar de llamar al backend correctamente. Eso debido a que la variable `REACT_APP_API_URL` fue configurada en Vercel sin el prefijo `https://`, lo que hizo que el navegador la interpretara como una ruta relativa y la concatenara al dominio del frontend. Adicionalmente, React compila las variables de entorno en el momento del build, por lo que un cambio en las variables requiere un nuevo despliegue.

Para solucionarlo se corrigio el valor de `REACT_APP_API_URL` para incluir el protocolo completo (`https://`) y se ejecuto un redeploy manual en Vercel para que el build tomara el valor correcto.

Po lo cual nos pudimos dar cuenta de que en Create React App, las variables de entorno con prefijo `REACT_APP_` se alistan en el bundle en tiempo de compilación. Cualquier cambio en estas variables requiere un nuevo build. Siempre verificar que las URLs incluyan el protocolo y no dependan de rutas relativas.

Lección 4 - Gestión de versiones del código desplegado

Después de agregar nuevas rutas al backend (endpoint [/api/appointments/psychologist](#)), la aplicación retornaba 404 "Ruta no encontrada" a pesar de que el código en el repositorio era



correcto.

Eso debido a que el servicio de Render no había red desplegado automáticamente después del push. El servidor estaba ejecutando una versión anterior del código sin las nuevas rutas. Esto ocurre cuando el auto-deploy falla silenciosamente o cuando el trigger del webhook no se activa. como solución se reviso el estado del último deploy en el dashboard de Render y se ejecuto un "Manual Deploy" para forzar el red despliegue con el código mas reciente. como aprendizaje tenemos que tras agregar nuevas rutas o funcionalidades al backend, siempre verificar en el panel de la plataforma de despliegue que el nuevo deploy se ejecutó exitosamente y que el servidor esta corriendo la versión correcta del código.

Lección 5 - Hashes de contraseñas en datos de prueba

Durante la ejecución de las pruebas, paso lo siguiente, los usuarios insertados directamente en la base de datos mediante scripts SQL no podían iniciar sesión a pesar de tener verificado = 1 y los datos correctos. El backend retornaba 401 "Credenciales incorrectas". La cause era que el hash de bcrypt utilizado en el script SQL de inserción no correspondía a la contraseña configurada en las pruebas. El hash fue escrito manualmente sin verificación, y bcrypt es un algoritmo unidireccional que no puede verificarse sin ejecutarlo. para solucionarlo se genero el hash correcto ejecutando `bcrypt.hash('Admin1234*', 10)` directamente con Node.js en el entorno local, obteniendo el valor real del hash. Luego se actualizo la base de datos con el hash correcto mediante un UPDATE. Aprendizaje: Los hashes de bcrypt nunca deben escribirse manualmente en scripts. Siempre generarlos programáticamente con la misma librería y versión que usa la aplicación y se debe documentar el comando exacto para generarlos.



Leccion 6 - Plan gratuito de Render (cold start)

Cuando un usuario hace una petición inicial, se evidencia que la aplicación tarda hasta 30 segundos en responder esa primera vez, posterior a un periodo de inactividad. Eso se debe a que el plan gratuito de Render suspende el servidor tras 15 minutos sin recibir peticiones. La primera solicitud debe "despertar" el servidor, lo que toma entre 20 y 30 segundos. Esta eventualidad se documentó en el manual de usuario y en el README. En las pruebas automatizadas se implementó el comando [cy.wakeUpBackend\(\)](#) que realiza múltiples intentos de conexión al inicio de cada suite de pruebas para garantizar que el servidor este activo antes de ejecutar los casos de prueba. como enseñanza podemos decir que en entornos de producción con plan gratuito, el cold start es una limitación conocida que debe comunicarse claramente a los usuarios. Para ambientes de prueba o producción de mayor exigencia, se debería considerar un plan de pago.